Week 4 →

learning Policies directly ⇒

Constraints on the policy parameterization

$$\pi(a|s,\theta) \geq 0 \quad \text{for all } a \in \mathcal{A} \text{ and } s \in \mathcal{S}$$

$$\sum_{a \in \mathcal{A}} \pi(a|s,\theta) = 1 \quad \text{for all } s \in \mathcal{S}$$

The Softmax policy parameterization →

$$\pi(a|s,\theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,a,\theta)}} \leftarrow \text{action preference!}$$

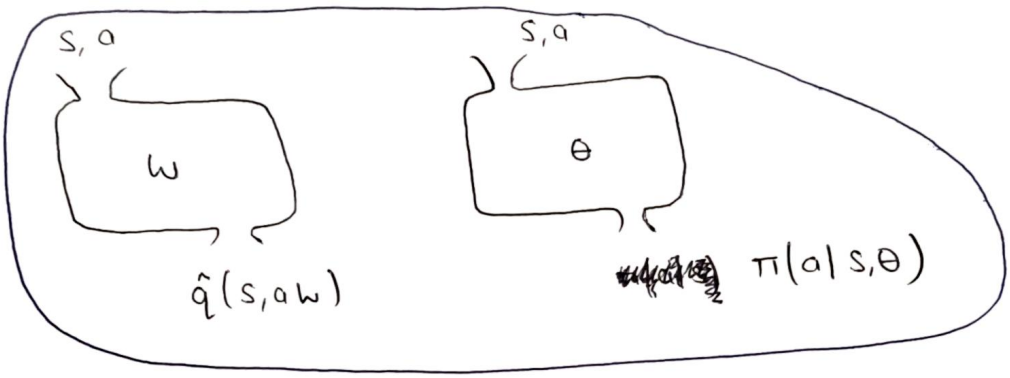
A higher preference for a particular action in a state, means that the action is more likely to be started.

Action preferences and action values are not same. Preferences indicate how much the agent prefers each action but they are not ~~summaries~~ summaries of future reward. Only the relative difference b/w preferences are important!

An ϵ-greedy policy derived from action values can behave very differently than a softmax policy over action pre-ferences.

Advantages of policy parameterization :-

1) agent can make its policy more greedy over time autonomously.



- A softmax policy can adequately approximate a deterministic policy by making one action preference very large.

- In tabular setting, we learn there's always a deterministic optimal policy. However in function approximation, we may not be able to represent this deterministic policy. Instead, the optimal approximate policy might be a stochastic policy!

- Stochastic policies might be better than deterministic policies under function approximation } why?

  why?

- Sometimes policies are less complicated than the value function!

Objective of learning policies →

- In the average reward formulation, here, we maximise the long term average of the reward. The appropriate return here is the sum of the differences b/w the immediate reward and its average. Because the long term averages subtracted, this sum is finite even without discounting.

- Our Aim :- to find a way to directly optimize the parameters of the policy!

Average Reward objective →

$$r(\pi) = \sum_s \mu(s) \sum_a \pi(a|s,\theta) \sum_{s',r} p(s',r|s,a) \times r$$

- Previously, we estimated the average reward to learn action values in an average reward variant of SARSA.
- Now our aim is to learn a policy that directly optimises the average reward.

- our basic approach will be to estimate the gradient of the objective with respect to the policy parameters and adjust the parameters based on the estimate

$\{\nabla r(\pi)\}$ ⇒ The class of methods they use this idea are often referred to as policy gradient methods!

- Upto now, we were using Generalized policy iteration framework to approximate action values. Then we use these approximate values indirectly to infer a good policy. Now we are learning the policies directly!

- Before we were minimizing the mean square value error, and now we are maximizing the objective.

Challenge of policy gradient methods →

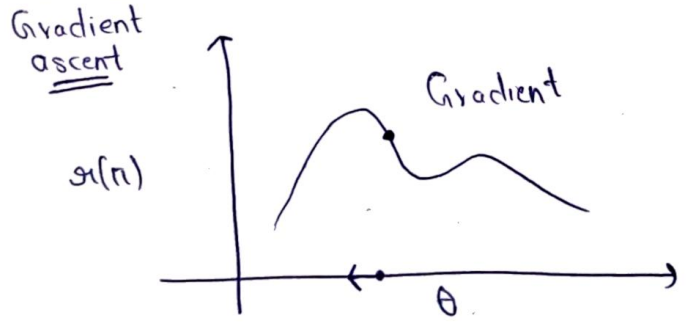
$$\nabla_\theta \eta(\pi) = \nabla_\theta \left\{ \sum_s \mu(s) \sum_a \pi(a|s,\theta) \sum_{s',r} p(s',r|s,a) \cdot r \right\}$$

$\mu(s)$ ↑ depends on θ

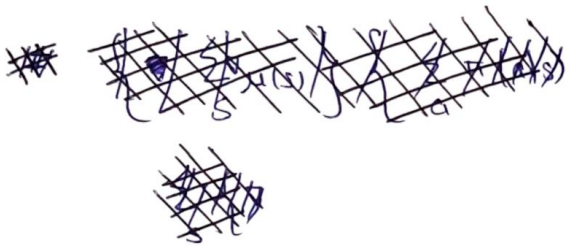
$$\nabla_w \overline{VE} = \nabla_w \left\{ \sum_s \mu(s) \left[v_\pi(s) - \hat{v}(s,w) \right]^2 \right\}$$

$\mu(s)$ ↑ Independent of w

Policy Gradient Theorem →



$$\nabla r(\pi) = \nabla \Big\{ \sum_{s} \mu(s) \sum_{a} \pi(a|s) \sum_{s', r} p(s', r | s, a) \times r \Big)$$



$$= \sum_{s} \nabla \{\mu(s)\} \cdot \sum_{a} \pi(a|s) \sum_{s', r} p(s', r) s, a) \times r$$

$$+ \sum_{s} \mu(s) \cdot \nabla \Big\{ \sum \pi(a|s) \sum_{s', r} p(s', r) s, a) \times r \Big\}$$

$$\Rightarrow \nabla r(\pi) = \sum_{s} \mu(s) \sum_{a} \nabla \pi(a | s, \theta) q_{\pi}(s, a)$$

} Policy gradient theorem gives us this simpler expression!

✱ The gradient of the policy tells you how to adjust the parameters to increase the probability of certain action.

Estimating the policy gradient :→

Here we will drive a sample based estimate for the gradient of the average reward objective

we want to derive gradient descent algorithm for our policy!

We have our objective and its gradient due to the policy gradient theorem.

$$\nabla r(\pi) = \sum_s \mu(s) \sum_a \nabla \{\pi(a|s,\theta) \cdot q_\pi(s,a)\}$$

Getting stochastic samples of the gradient ⟶

Computing the sum over states is really impractical. But we can do the same thing we did when deriving our stochastic gradient descent rule for our policy evaluation. We simply make updates from states we observe while following policy pi(π).

Stochastic gradient descent update for the policy parameters —

$$\theta_{t+1} = \theta_t + \alpha \sum_a \nabla \{ \pi(a|s_t,\theta_t) \cdot q_\pi(s_t,a) \}$$

Unbiasedness of the Stochastic Samples →

$$\nabla \mathcal{J}(\pi) = \sum_s \mu(s) \sum_a \nabla \{ \pi(a|s,\theta)\, q_\pi(s,a) \}$$

$$= \mathbb{E}_\mu \left[\sum_a \nabla \{ \pi(a|s,\theta)\, q_\pi(s,a) \} \right]$$

$$\Rightarrow \sum_a \nabla \{ \pi(a|s,\theta)\, q_\pi(s,a) \}$$

$$= \sum_a \pi(a|s,\theta) \times \frac{1}{\pi(a|s,\theta)} \nabla \{ \pi(a|s,\theta)\, q_\pi(s,a) \}$$

$$= \mathbb{E}_\pi \left[\frac{\nabla \{ \pi(A|s,\theta) \}}{\pi(A|s,\theta)} \cdot q_\pi(s,A) \right]$$

$$\Rightarrow \theta_{t+1} \doteq \theta_t + \alpha \cdot \frac{\nabla \{ \pi(A_t|s_t,\theta_t) \}}{\pi(A_t|s_t,\theta_t)} \cdot q_\pi(s_t,A_t)$$

$$\theta_{t+1} \doteq \theta_t + \alpha \nabla \{ \ln(\pi(A_t|s_t,\theta_t)) \} \cdot q_\pi(s_t,A_t)$$

- $\nabla \{ \ln(\pi(A_t|s_t,\theta_t)) \}$: gradient of the policy!
- $q_\pi(s_t,A_t)$: estimate of the differential values.

We know the policy and the parameterization, and so compute its gradient →

These action values can be approximated in a variety of ways. for e.g we could use a TD algorithm that learns a differential action values.

Actor - Critic Algorithm →

Here, the parameterized policy plays the role of an actor, while the value function plays the role of Critic, evaluating the actions selected by the actor.

$$\theta_{t+1} \doteq \theta_t + \alpha \nabla \left[\ln \{ \pi(A_t | S_t, \theta_t) \} \right] \cdot q_\pi(S_t, A_t)$$

but we don't have access to q_π, so we have to approximate it! We can do the usual TD thing, the one-step, bootstrap return.

$$\theta_{t+1} \doteq \theta_t + \alpha \nabla \{ \ln(\pi | A_t | S_t, \theta_t) \} \underbrace{[R_{t+1} - \bar{R} + \hat{v}(S_{t+1}, w)]}$$

This is the critic part. The critic provides immediate feedback. To train the critic, we can use any state value learning algorithm. We will use average reward version of semi gradient TD.

$$\underbrace{[R_{t+1} - \bar{R} + \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)]}_{\text{TD error } \delta}$$

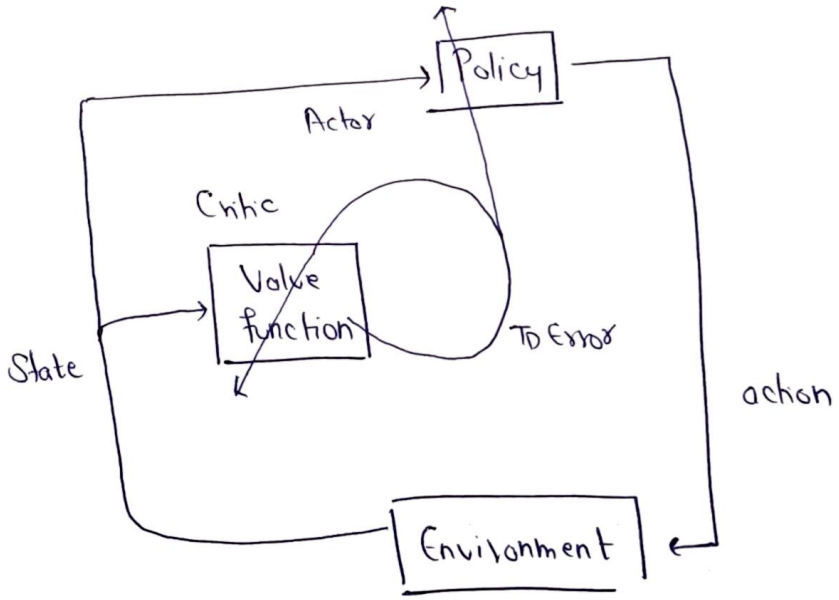
Does not effect the expected update!

Reduces the update variance

$$\mathbb{E}_\pi \left[\nabla \ln \pi(A_t | S_t, \theta_t) \left[R_{t+1} - \bar{R} + \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w) \right] \middle| S_t = s \right]$$

$$= \mathbb{E}_\pi \left[\nabla \ln \pi(A_t | S_t, \theta_t) \left[R_{t+1} - \bar{R} + \hat{v}(S_{t+1}, w) \right] \middle| S_t = s \right]$$

$$- \underbrace{\mathbb{E}_\pi \left[\nabla \ln \pi(A_t | S_t, \theta_t) \, \hat{v}(S_{t+1}, w) \middle| S_t = s \right]}_{= 0}$$



Actor Critic (Continuing), for estimating $\pi_\theta \simeq \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\theta)$

Input: a differentiable state value function parameterization $\hat{v}(s,w)$

Initialize $\bar{R} \in \mathbb{R}$ to 0

Initialize state value weights $w \in \mathbb{R}^d$ and policy parameters $\theta \in \mathbb{R}^{d'}$ (e.g. to 0)

Algorithm parameters: $\alpha^w > 0$, $\alpha^\theta > 0$, $\alpha^{\bar{R}} > 0$

Initialize $S \in \mathcal{S}$

Loop forever (for each time stamp):

$A \sim \pi(\cdot|S,\theta)$

Take Action A, observe R and S'

$\delta \leftarrow R - \bar{R} + \hat{v}(S',w) - \hat{v}(S,w)$

$\bar{R} \leftarrow \bar{R} + \alpha^{\bar{R}} \delta$

$w \leftarrow w + \alpha^w \delta \nabla \hat{v}(S,w)$

$\theta \leftarrow \theta + \alpha^\theta \delta \nabla \ln \pi(A|S,\theta)$

$S \leftarrow S'$

Actor Critic with Softmax Policies →

$$\theta \leftarrow \theta + \alpha^{\theta} \delta \nabla \ln \pi(A|S,\theta)$$

$$\pi(a|s,\theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}}$$

$$\Rightarrow \quad \hat{v}(s,w) \doteq w^T x(s)$$

$$h(s,a,\theta) \doteq \theta^T x_h(s,a)$$

$$x_h(s,a) = \begin{bmatrix} x_0(s) \\ x_1(s) \\ x_2(s) \\ x_3(s) \\ \vdots \\ \vdots \\ \vdots \end{bmatrix} \begin{matrix} \} a_0 \\ \\ \\ \\ \} a_1 \end{matrix}$$

$$\Rightarrow \quad \nabla \hat{v}(s,w) = x(s)$$

$$\nabla \ln \pi(A|S,\theta) = \frac{\nabla \pi(A|S,\theta)}{\pi(A|S,\theta)}$$

$$= \frac{1}{\pi(A|S,\theta)} \nabla \left\{ \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}} \right\}$$

$$= \frac{1}{\pi(A|S,\theta)} \nabla \left\{ \frac{e^{\theta^T x_h(s,a)}}{\sum_{b \in \mathcal{A}} e^{\theta^T x_h(s,b)}} \right\}$$

$$= \frac{1}{\pi(A|S,\theta)} \left\{ x_h(s,a) \frac{e^{\theta^T x_h(s,a)}}{\sum_{b \in \mathcal{A}} e^{\theta^T x_h(s,b)}} - \frac{e^{\theta^T x_h(s,a)} \sum_{b \in \mathcal{A}} x_h(s,b) e^{\theta^T x_h(s,b)}}{\left(\sum_{b \in \mathcal{A}} e^{\theta^T x_h(s,b)} \right)^2} \right.$$

$$= x_h(s,a) - \sum_{b \in \mathcal{A}} \left\{ x_h(s,b) e^{\theta^T x_h(s,b)} \right.$$

$$\nabla \ln\left(\pi(a|s,\theta)\right) = x_\pi(s,a) - \sum_b \pi(b|s,\theta)\, x_\pi(s,b)$$

Gaussian Policy →

$$\pi(a|s,\theta) \doteq \frac{1}{\sigma(s,\theta)\sqrt{2\pi}} \exp\left(-\frac{(a-\mu(s,\theta))^2}{2\sigma(s,\theta)^2}\right)$$

$$\mu(s,\theta) \doteq \theta_\mu^T x(s)$$
$$\sigma(s,\theta) \doteq \exp\left(\theta_\sigma^T x(s)\right)$$
$$\theta \doteq \begin{bmatrix} \theta_\mu \\ \theta_\sigma \end{bmatrix}$$

$$\Rightarrow \quad \nabla \ln \pi(a|s,\theta_\mu) = \frac{1}{\sigma(s,\theta)^2}\left(a-\mu(s,\theta)\right) x(s)$$

$$\nabla \ln \pi(a|s,\theta_\sigma) = \left(\frac{(a-\mu(s,\theta))^2}{\sigma(s,\theta)^2} - 1\right) x(s)$$